

Module 4 Project Rubric

PromptLab | Production-Ready Backend | AIE 500 - 10x Coding with AI

C2.4-C2.8, plus final judgment on C2.1 and C2.2. All eight C2 criteria must be Met.

Implement one of the two specified features using test-first increments, then verify, refactor, automate checks and containerize the backend.

Evidence to prepare

- Implemented first feature with genuine failing-test commits before implementation and meaningful pytest-cov coverage of at least 80%.
- docs/refactor-note.md naming the smell, before/after commit hashes and a green suite on both sides with unchanged public behavior.
- .github/workflows/ci.yml plus actual run logs and docs/ci-gate-evidence.md proving a clean success and a deliberately broken-test failure.
- backend/Dockerfile, a Compose configuration, build/run evidence and working Docker instructions in the README.
- Both specifications and the API reference/docstrings updated for the implemented feature; the second feature remains explicitly planned.

MUST PASS: C2.4 - Your tests came first; C2.7 - Your pipeline actually gates

A Not Yet on this requirement returns the submission without reviewing the remaining criteria in this module.

How grading works

Every criterion is **Met** or **Not Yet**. There are no points, percentages, weights or bonus credit. All criteria in a competency must be Met; nothing is averaged.

Three attempts are available per module evidence group. Returned submissions do not consume an attempt, but require a revision plan. Re-assessment covers Not Yet criteria and any evidence affected by the changes. Follow the Syllabus for policy and course instructions for submission timing.

Returned before assessment: Missing deliverables, failing provided tests, an application that does not run or committed secrets. Rotate exposed credentials before resubmitting. Work belonging to another person or reconstructed logs/history are referred for academic review.

Module 4 | Assessment criteria (1/3)

C2 - Specification-Driven Development & Automated Quality Engineering

C2.4-C2.8, plus final judgment on C2.1 and C2.2. All eight C2 criteria must be Met.

Criterion	Where your evidence lives	What earns "Met"	Common mistakes
C2.4 Your tests came first (MUST PASS)	Commit history for your Module 4 feature	For each part of the feature, a commit containing a failing test precedes the commit containing its implementation. The Red-Green-Refactor rhythm is visible.	- All tests in one commit after the code. - Squashed history, so ordering can't be seen. - Tests committed first that assert nothing capable of failing.
C2.5 Spec, tests and code line up	tests/, coverage report, specs/	Coverage at least 80% measured with pytest-cov. Every endpoint in your specs and API reference has at least one passing test for the success path and one for a documented failure case. Storage, utils and model validation are covered.	- Endpoints built but not tested, or tested but not specified. - Coverage reached by tests that run code without asserting anything. - Failure cases specified but never tested.
C2.6 Your refactor changed nothing visible	docs/refactor-note.md, before/after commits	A named code smell removed, the two commit hashes given, suite green on both sides, public interface and observable behaviour unchanged.	- Not naming the smell, or fixing something other than the one named. - Changing behaviour at the same time. - Editing tests to accommodate the refactor instead of using them to check it.

Threshold and evidence. The assessment threshold is 80%, not a higher tutorial exercise threshold. A workflow file alone does not establish a successful run or a failing quality gate. Keep actual runs and real commit order.

Module 4 | Assessment criteria (2/3)

C2 - Specification-Driven Development & Automated Quality Engineering

C2.4-C2.8, plus final judgment on C2.1 and C2.2. All eight C2 criteria must be Met.

Criterion	Where your evidence lives	What earns "Met"	Common mistakes
C2.7 Your pipeline actually gates (MUST PASS)	.github/workflows/ci.yml, run logs, docs/ci-gate-evidence.md	Triggers on push and pull request, runs linting and tests with coverage, fails the build below 80%, passes on a clean clone, and demonstrably fails when a test is deliberately broken.	- A workflow file with no successful run recorded. - A pipeline that runs tests but doesn't fail the build when they fail. - Passing locally but failing on a clean clone because config was never committed.
C2.8 Your container builds and runs	backend/Dockerfile, docker-compose.yml, build output	The image builds from committed files with no manual steps. The container serves the API. docker-compose up --build brings up a working local environment, and the README explains Docker usage.	- An image that builds but a container that exits or serves nothing. - A Dockerfile depending on files not in the repository. - A build needing undocumented manual steps.

Prove both automation and runtime behavior. Keep a genuine CI run showing that the deliberately broken test fails the build, followed by recovery. Build and run the container from committed files and follow the README from a clean copy. A passing image build alone does not show that the API is served.

Module 4 | Assessment criteria (3/3)

C2 - Specification-Driven Development & Automated Quality Engineering

C2.4-C2.8, plus final judgment on C2.1 and C2.2. All eight C2 criteria must be Met.

Criterion	Where your evidence lives	What earns "Met"	Common mistakes
C2.1 Your specs are specific enough Final judgment in Module 4	specs/prompt-versions.md, specs/tagging-system.md	Each spec has overview and goals, user stories with acceptance criteria, data model changes, API endpoints with request and response shapes, and specified error conditions and edge cases. Every endpoint has at least one acceptance criterion you could write a test from directly.	- Requirements that name a feature without describing behaviour. - Specifying the happy path and leaving errors unstated. - Criteria you can't test, like "handles input correctly".
C2.2 Your docs match your code Final judgment in Module 4	README.md, docstrings in models.py / api.py / storage.py / utils.py, docs/API_REFERENCE.md	Docstrings on every function and class across all four source modules, with parameters, returns and exceptions matching the implementation. API reference documents every endpoint that exists - including PATCH and your Module 4 feature - with request examples, sample responses and error formats. README setup steps work on a clean clone.	- API reference describing an earlier draft of the spec. - Docs not updated after Module 4's feature work. - Setup steps missing an environment variable or dependency. - Docstrings that restate the function name.

Finalizing the provisional criteria. The C2.1 and C2.2 rows above are repeated from Module 3 for final review. Check both specs, the implemented first feature and current documentation. Implement the remaining feature in Module 5 and keep affected documentation and tests synchronized.

Source: PromptLab Grading Rubric v2.1. Original Modules 1-5 map to current Modules 2-6. Criterion standards are retained; module/tool references are updated. The Syllabus governs policy.